

Pandas I — Series, DataFrames & Reading Real Data

By the end of today you will be able to load a real dataset from a CSV file, inspect it confidently, and understand its structure. Pandas is the single most important library in data science — it is what you use to work with real-world messy data before feeding it into any ML model. NumPy is the engine underneath. Pandas is the steering wheel.

SECTION 01 Install Pandas — Do This First

Pandas is not built into Python. Install it once and you are set for the entire program.

✂ Installation (one command)

Mac users — activate your venv first, then:

```
source venv/bin/activate  
pip install pandas
```

Windows users:

```
pip install pandas
```

Linux users:

```
pip3 install pandas
```

Install both together for today and future days:

```
pip install pandas kaggle
```

Verify it worked:

```
python3 -c "import pandas as pd; print(pd.__version__)"
```

You should see something like: 2.1.4

The alias `pd` is the universal standard — always use it, never change it.

SECTION 02 What is Pandas and Why Does Every Data Scientist Use It?

You have been working with NumPy arrays — which are fast and powerful but have one big limitation. They have no labels. Column 0 means nothing without context. You have to remember what each position represents.

Imagine your NumPy array of student data:

```
[[25, 35, 81], [47, 82, 44], [31, 28, 55]]
```

What does column 0 mean? Column 1? Column 2?

You have to memorise it or check your notes every time.

With Pandas, the same data looks like this:

	age	income	spending_score
0	25	35	81
1	47	82	44
2	31	28	55

Every column has a name. Every row has an index. The data explains itself.

Pandas = NumPy arrays + labels + a huge toolbox for data manipulation.

Think of NumPy as a calculator — powerful but no labels.

Think of Pandas as Excel, but programmable, 1000x faster, and able to handle millions of rows without crashing.

If NumPy is the engine of a car, Pandas is the full dashboard with dials, labels, GPS, and a steering wheel.

SECTION 03 The Series — A Single Column of Data

Series

What it is: A one-dimensional labelled array. Like a single column in a spreadsheet. Every value has an index label. It can hold any data type: numbers, strings, booleans, dates.

Real-life analogy: Think of a single column in your Excel sheet. The column is called 'Student Name' and each cell has a row number on the left. That is exactly a Pandas Series — a named column with row labels.

In Python:

```
s = pd.Series([85, 92, 78, 95],  
index=['Rahul', 'Priya', 'Arjun', 'Sneha'])
```

Creating a Series

```

import pandas as pd
import numpy as np

# From a list - default index is 0, 1, 2, 3...
marks = pd.Series([85, 92, 78, 95, 88])
print(marks)
print("")

# With custom index labels
marks_named = pd.Series(
    [85, 92, 78, 95, 88],
    index=["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]
)
print(marks_named)
print("")

# From a dictionary - keys become index automatically
attendance = pd.Series({
    "Rahul" : 92,
    "Priya" : 85,
    "Arjun" : 78,
    "Sneha" : 96
})
print(attendance)

```

Output:

```

0      85
1      92
2      78
3      95
4      88
dtype: int64

Rahul      85
Priya      92
Arjun      78
Sneha      95
Mohammed   88
dtype: int64

Rahul      92
Priya      85
Arjun      78
Sneha      96
dtype: int64

```

Accessing values in a Series

```

marks = pd.Series(
    [85, 92, 78, 95, 88],
    index=["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]
)

```

```

# Access by label (name)
print(marks["Rahul"])      # 85
print(marks["Priya"])      # 92

# Access by position (number)
print(marks.iloc[0])       # 85   (first item)
print(marks.iloc[-1])      # 88   (last item)

# Slice by label
print(marks["Rahul":"Arjun"]) # Rahul to Arjun inclusive

# Boolean filtering – same as NumPy
print(marks[marks > 85])    # Only scores above 85

```

Output:

```

85
92
85
88
Rahul      85
Priya      92
Arjun      78
dtype: int64
Priya      92
Sneha      95
Mohammed   88
dtype: int64

```

Useful Series operations

```

marks = pd.Series([85, 92, 78, 95, 88, 70, 99, 63])

# Quick statistics
print(marks.mean())      # 83.75
print(marks.median())    # 86.5
print(marks.max())       # 99
print(marks.min())       # 63
print(marks.std())       # 11.67
print(marks.count())     # 8   (number of non-null values)

# .describe() – all stats in one shot
print(marks.describe())

# Value counts – very useful for categorical data
grades = pd.Series(["A", "B", "A", "C", "A", "B", "F", "A"])
print(grades.value_counts())

```

Output:

```

83.75

```

```
86.5
99
63
11.67
8
count      8.000
mean       83.750
std        11.670
min        63.000
25%        77.500
50%        86.500
75%        91.750
max        99.000
dtype: float64

A      4
B      2
C      1
F      1
Name: count, dtype: int64
```

SECTION 04 The DataFrame — The Heart of Pandas

DataFrame

What it is: A two-dimensional labelled table with rows and columns. Like a spreadsheet or a database table. Each column is a Series. All columns share the same index. This is how you will work with EVERY dataset in this program.

Real-life analogy: Think of a complete Excel spreadsheet. Rows represent individual records (one customer, one transaction, one student). Columns represent attributes (name, age, city, score). Each cell has a unique address: row label + column name. That is a DataFrame.

In Python: `df = pd.DataFrame(data)` — creates a table from a dictionary or list of dicts

Creating a DataFrame

The most common way to create a DataFrame is from a dictionary where keys are column names and values are lists of data.

```
import pandas as pd

# Method 1: From a dictionary (most common)
data = {
    "name"      : ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"],
    "age"       : [22, 21, 23, 20, 24],
    "city"      : ["Bengaluru", "Mumbai", "Delhi", "Chennai",
                  "Hyderabad"],
```

```

    "marks"      : [85, 92, 78, 95, 88],
    "passed"     : [True, True, True, True, True]
}

df = pd.DataFrame(data)
print(df)

```

Output:

	name	age	city	marks	passed
0	Rahul	22	Bengaluru	85	True
1	Priya	21	Mumbai	92	True
2	Arjun	23	Delhi	78	True
3	Sneha	20	Chennai	95	True
4	Mohammed	24	Hyderabad	88	True

```

# Method 2: From a list of dictionaries
# (This is how data often comes from APIs and databases)
records = [
    {"name": "Rahul",      "score": 85, "grade": "B"},
    {"name": "Priya",      "score": 92, "grade": "A"},
    {"name": "Arjun",      "score": 78, "grade": "B"},
]
df2 = pd.DataFrame(records)
print(df2)

```

Output:

	name	score	grade
0	Rahul	85	B
1	Priya	92	A
2	Arjun	78	B

SECTION 05 Reading Real Data — Loading CSV, Excel & JSON

In real data science work, you never type data manually. You load it from files. The most common format is CSV (Comma Separated Values). Let us learn how to load real datasets.

What is a CSV file?

A CSV is a plain text file where data is separated by commas. Every line is a row. The first line is usually the header (column names).

```

name,age,city,marks
Rahul,22,Bengaluru,85
Priya,21,Mumbai,92

```

Arjun,23,Delhi,78

Almost every real-world dataset comes in CSV format.
Excel files, database exports, Kaggle datasets — all of them.

Downloading your first real dataset — Titanic

The Titanic dataset is the most famous beginner dataset in data science. It contains information about 891 passengers on the Titanic and whether they survived. Let us download and load it.

✂ Get the Titanic dataset — two options:

Option A (Easiest — direct download):

Go to this URL in your browser and save the file:

<https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv>

Right-click → Save As → save as titanic.csv in your Day_7 folder

Option B (Using Python):

```
import urllib.request
url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
urllib.request.urlretrieve(url, 'titanic.csv')
print('Downloaded!')
```

Put titanic.csv in the same folder as your Python file.

Then load it with: `df = pd.read_csv('titanic.csv')`

pd.read_csv() — loading a CSV file

`read_csv()` is the most used function in all of Pandas. It reads a CSV file and returns a DataFrame.

```
import pandas as pd

# Load the Titanic CSV file
df = pd.read_csv("titanic.csv")

# Show the first 5 rows
print(df.head())
```

Output:

	PassengerId	Survived	Pclass	...	Age	SibSp	Parch	Fare
0	1	0	3	...	22.0	1	0	7.2500
1	2	1	1	...	38.0	1	0	71.2833
2	3	1	3	...	26.0	0	0	7.9250
3	4	1	1	...	35.0	1	0	53.1000
4	5	0	3	...	35.0	0	0	8.0500

[5 rows x 12 columns]

Parameter	What it does
<code>pd.read_csv('file.csv')</code>	Basic load. Assumes comma separator, first row is header.
<code>sep=';'</code>	Use if values are separated by semicolons instead of commas.
<code>header=None</code>	Use if file has no header row. Pandas assigns 0,1,2 as column names.
<code>index_col='name'</code>	Set a specific column as the row index instead of 0,1,2,3.
<code>usecols=['a','b']</code>	Load only specific columns. Saves memory on large files.
<code>nrows=100</code>	Load only the first 100 rows. Useful for previewing huge files.
<code>encoding='utf-8'</code>	Specify file encoding. Use if you get a <code>UnicodeDecodeError</code> .

Reading other file formats

```
import pandas as pd

# Excel file (.xlsx or .xls)
# pip install openpyxl (needed for xlsx files)
df_excel = pd.read_excel("data.xlsx", sheet_name="Sheet1")

# JSON file (common from APIs and web scraping)
df_json = pd.read_json("data.json")

# From a URL directly (no download needed!)
url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df_url = pd.read_csv(url)
print(f"Loaded {len(df_url)} rows directly from the internet")

# Save a DataFrame back to CSV
df.to_csv("my_results.csv", index=False) # index=False - don't save row
numbers
```

Output:

Loaded 891 rows directly from the internet

SECTION 06 Inspecting a DataFrame — The First Thing You Always Do

Every single time you load a new dataset, you run these inspection commands before doing anything else. This is not optional. Skipping this step leads to bugs and wrong models.

The first 10 minutes with any new dataset: load it, run all these commands, understand what you have. Only then do you start cleaning or modelling.

The essential inspection commands

```
import pandas as pd

df = pd.read_csv("titanic.csv")

# 1. Shape — how big is this dataset?
print(df.shape)          # (891, 12) — 891 rows, 12 columns

# 2. head() — see the first 5 rows (or any number you specify)
print(df.head())          # first 5
print(df.head(10))        # first 10

# 3. tail() — see the last 5 rows
print(df.tail())          # last 5

# 4. info() — THE MOST IMPORTANT COMMAND
# Shows: column names, data types, non-null counts
# Immediately reveals missing data and wrong data types
print(df.info())
```

Output:

```
(891, 12)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   PassengerId     891 non-null   int64
 1   Survived        891 non-null   int64
 2   Pclass          891 non-null   int64
 3   Name            891 non-null   object
 4   Sex             891 non-null   object
 5   Age            714 non-null   float64  <-- 177 MISSING!
 6   SibSp          891 non-null   int64
 7   Parch          891 non-null   int64
 8   Ticket         891 non-null   object
 9   Fare           891 non-null   float64
10   Cabin          204 non-null   object  <-- 687 MISSING!
11   Embarked       889 non-null   object  <-- 2 missing
```

Look at what `df.info()` just revealed:

Age column — only 714 of 891 rows have a value. 177 are missing.

Cabin column — only 204 of 891 rows have a value. 687 are missing (77%).

Embarked col — 2 rows are missing.

If you had gone straight to building a model without checking this, it would either crash or silently give you terrible results.
`df.info()` is what saves you.

```
# 5. describe() - statistics for all numeric columns
print(df.describe())

# 6. columns - list all column names
print(df.columns.tolist())

# 7. dtypes - data type of each column
print(df.dtypes)

# 8. Check missing values specifically
print(df.isnull().sum())    # Count of missing values per column

# 9. Check missing as percentage
missing_pct = (df.isnull().sum() / len(df)) * 100
print(missing_pct.round(1))
```

Output:

	PassengerId	Survived	Pclass	Age	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	32.204208
std	257.353842	0.486592	0.836071	14.526497	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	512.329200

PassengerId	0
Survived	0
Age	177
Cabin	687
Embarked	2

SECTION 07 Selecting Rows and Columns — loc and iloc

Once you have loaded data, you need to pick specific parts of it. Pandas gives you two main tools for this: `loc` for label-based selection and `iloc` for position-based selection. This confuses most beginners at first but becomes second nature quickly.

Simple rule to remember:

loc — use LABELS (names of rows and columns)
iloc — use INTEGERS (position numbers, like array indexing)

Selecting columns

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Select ONE column — returns a Series
ages = df["Age"]
print(type(ages))          # <class 'pandas.core.series.Series'>
print(ages.head())

# Select MULTIPLE columns — returns a DataFrame
# Note the double square brackets [[ ]]
subset = df[["Name", "Age", "Survived"]]
print(subset.head())

# Select all columns EXCEPT some
no_id = df.drop(columns=["PassengerId", "Ticket", "Cabin"])
print(no_id.shape)         # (891, 9)
```

Output:

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: Age, dtype: float64
```

```
      Name    Age  Survived
0  Braund, Mr.   22.0         0
1  Cumings, Mrs. John Bradley  38.0         1
2  Heikkinen, Miss.   26.0         1
(891, 9)
```

Selecting rows with loc — by label

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Single row by index label
print(df.loc[0])          # First row (all columns)

# Range of rows by label
print(df.loc[0:4])        # Rows 0 to 4 INCLUSIVE (loc includes end)

# Specific row AND specific columns
```

```
print(df.loc[0, "Name"])          # First passenger's name
print(df.loc[0, ["Name", "Age", "Survived"]])

# Multiple rows and multiple columns
print(df.loc[0:3, ["Name", "Survived", "Fare"]])
```

Output:

```
PassengerId      1
Survived         0
Name      Braund, Mr.  Owen Harris Braund
Age                22
...

Braund, Mr.  Owen Harris Braund
Name      Braund, Mr.  Owen Harris Braund
Age                22.0
Survived          0

      Name  Survived  Fare
0  Braund, Mr.      0   7.2500
1  Cumings, Mrs.    1  71.2833
2  Heikkinen, Miss.  1   7.9250
3  Futrelle, Mrs.   1  53.1000
```

Selecting rows with iloc — by position number

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Single row by position
print(df.iloc[0])          # First row
print(df.iloc[-1])         # Last row

# Range of rows by position (end is EXCLUDED, like Python slicing)
print(df.iloc[0:5])        # First 5 rows (0,1,2,3,4)

# Specific rows and columns by position
print(df.iloc[0:3, 0:4])    # First 3 rows, first 4 columns

# Every other row
print(df.iloc[::2].head())

# Last 10 rows
print(df.iloc[-10:])
```

Task	Use loc	Use iloc
Get one row	df.loc[0]	df.iloc[0]
Get one column	df.loc[:, 'Age']	df.iloc[:, 5]
Get a cell	df.loc[2, 'Name']	df.iloc[2, 3]

Get first N rows	<code>df.loc[0:N-1]</code>	<code>df.iloc[:N]</code> or <code>df.head(N)</code>
Get last N rows	<code>df.loc[len-N:]</code>	<code>df.iloc[-N:]</code> or <code>df.tail(N)</code>

Key difference to remember:

`loc[0:4]` — gives rows 0, 1, 2, 3, 4 (INCLUDES the end)

`iloc[0:4]` — gives rows 0, 1, 2, 3 (EXCLUDES the end, like Python slicing)

This inconsistency trips up almost everyone. `loc` is label-based so it includes both endpoints. `iloc` is position-based so it follows Python's standard convention.

Filtering rows with conditions — Boolean indexing in Pandas

This is one of the most important skills in data science. You filter rows by writing conditions on column values, just like `WHERE` in SQL.

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# All passengers who survived
survivors = df[df["Survived"] == 1]
print(f"Survivors: {len(survivors)}")

# All female passengers
women = df[df["Sex"] == "female"]
print(f"Women: {len(women)}")

# Passengers older than 40
older = df[df["Age"] > 40]
print(f"Over 40: {len(older)}")

# Combine conditions with & (AND) and | (OR)
# IMPORTANT: wrap each condition in parentheses ( )

# Women who survived
women_survived = df[(df["Sex"] == "female") & (df["Survived"] == 1)]
print(f"Women who survived: {len(women_survived)}")

# First class OR fare above 100
premium = df[(df["Pclass"] == 1) | (df["Fare"] > 100)]
print(f"Premium passengers: {len(premium)}")

# Passengers NOT in third class
not_third = df[df["Pclass"] != 3]
print(f"Not third class: {len(not_third)}")
```

Output:

```
Survivors: 342
Women: 314
Over 40: 163
Women who survived: 233
Premium passengers: 239
Not third class: 400
```

Common mistake with combined conditions:

WRONG: `df[df['Sex'] == 'female' and df['Survived'] == 1]`

RIGHT: `df[(df['Sex'] == 'female') & (df['Survived'] == 1)]`

Always use `&` for AND, `|` for OR — not Python's 'and'/'or' keywords.

Always wrap each condition in its own `()` parentheses.

This is the most common bug beginners write in Pandas.

SECTION 08 Adding and Modifying Columns

After loading data, you often need to create new columns derived from existing ones, or modify existing values. This is called feature engineering and it is a core skill in data science.

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Add a new column - fare in rupees (multiply by 90)
df["fare_inr"] = df["Fare"] * 90
print(df[["Fare", "fare_inr"]].head())

# Create a column based on a condition
df["is_child"] = df["Age"] < 18
print(df[["Name", "Age", "is_child"]].head(8))

# Create a category column using np.where
import numpy as np
df["age_group"] = np.where(df["Age"] < 18, "Child",
                           np.where(df["Age"] < 60, "Adult", "Senior"))
print(df["age_group"].value_counts())

# Rename a column
df = df.rename(columns={"Survived": "survived", "Pclass": "class"})
print(df.columns.tolist())
```

Output:

	Fare	fare_inr
0	7.25	652.50

```

1    71.28    6415.17
2     7.93     713.25

      Name    Age  is_child
0  Braund,  22.0    False
1  Cumings,  38.0    False
4   Allen,  35.0    False
5   Moran,   NaN    False

Adult      579
Child       83
Senior      22
Name: age_group, dtype: int64

```

SECTION 09 Quick Summary — Your Day 7 Cheat Sheet

Bookmark this. You will use every one of these commands in every data project.

Task	Code
Install Pandas	<code>pip install pandas</code>
Import	<code>import pandas as pd</code>
Load CSV	<code>df = pd.read_csv('file.csv')</code>
Load from URL	<code>df = pd.read_csv('https://...')</code>
Save to CSV	<code>df.to_csv('output.csv', index=False)</code>
First N rows	<code>df.head(N)</code>
Last N rows	<code>df.tail(N)</code>
Shape	<code>df.shape</code>
Column names	<code>df.columns.tolist()</code>
Data types	<code>df.dtypes</code>
Full info (BEST)	<code>df.info()</code>
Numeric stats	<code>df.describe()</code>
Missing values	<code>df.isnull().sum()</code>
Select one column	<code>df['Age']</code> or <code>df.Age</code>
Select many columns	<code>df[['Name', 'Age', 'Survived']]</code>
Select by label	<code>df.loc[row, col]</code>
Select by position	<code>df.iloc[row, col]</code>
Filter rows	<code>df[df['Age'] > 30]</code>
Multiple filters	<code>df[(cond1) & (cond2)]</code>

Add column	<code>df['new_col'] = values</code>
Rename columns	<code>df.rename(columns={'old':'new'})</code>
Value counts	<code>df['col'].value_counts()</code>
Unique values	<code>df['col'].unique()</code>

SECTION 10 Interview Prep — Questions to Answer After Today

These are questions that come up in every data science interview. Master them.

Q — What is the difference between a Pandas Series and a DataFrame?

A Series is a one-dimensional labelled array — like a single column in a spreadsheet. It has an index and values. A DataFrame is a two-dimensional labelled table with rows and columns — like a complete spreadsheet. You can think of a DataFrame as a dictionary of Series objects where each key is a column name and each value is a Series. In practice, a Series is what you get when you select one column from a DataFrame.

Q — What is the difference between loc and iloc in Pandas?

Both are used to select data from a DataFrame but they work differently. loc is label-based — you provide the actual row index labels and column names. If the index is 0 to 890, `df.loc[5]` gives row with label 5. iloc is position-based — you always use integer positions starting from 0, regardless of the actual index labels. `df.iloc[5]` always gives the 6th row. A subtle difference: loc includes the end of a range, iloc excludes it, consistent with Python slicing.

Q — What does `df.info()` tell you and why is it the first thing you run?

`df.info()` gives you a complete overview of a DataFrame: total number of rows, column names, data type of each column, and count of non-null values per column. The non-null count immediately reveals missing data — if a column has 714 non-null out of 891 rows, there are 177 missing values. It also shows data types, which reveals if a numeric column is being stored as text (object) or if dates are stored as strings. Running `df.info()` first prevents hours of debugging later because you understand your data before touching it.

Q — How do you filter rows in a Pandas DataFrame? Give an example.

You filter rows using Boolean indexing — passing a condition in square brackets. For example, `df[df['Age'] > 30]` returns all rows where the Age column is greater than 30. To combine multiple conditions, use `&` for AND and `|` for OR, and wrap each condition in parentheses: `df[(df['Age'] > 30) & (df['Sex'] == 'female')]`. The condition creates a Series of True/False values and Pandas returns only the rows where the value is True. This is equivalent to a WHERE clause in SQL.

SECTION 11 Words to Know

Term	Definition
Pandas	Python Data Analysis library. The main tool for working with tabular data.
Series	A 1D labelled array. Like a single column with an index.
DataFrame	A 2D labelled table with rows and columns. Like a spreadsheet.
Index	The row labels of a Series or DataFrame. Default is 0, 1, 2, 3...
CSV	Comma Separated Values. The most common file format for datasets.
pd.read_csv()	Load a CSV file into a DataFrame.
df.head()	Show the first 5 rows. Pass a number for more.
df.info()	Show column names, data types, and non-null counts.
df.describe()	Statistical summary of all numeric columns.
df.shape	Tuple of (rows, columns).
df.dtypes	Data type of each column.
df.isnull()	Returns True where values are missing (NaN).
NaN	Not a Number. Pandas' way of representing missing values.
loc	Select by label. df.loc[row_label, col_name]
iloc	Select by integer position. df.iloc[row_num, col_num]
Boolean indexing	Filtering rows using a True/False condition.
value_counts()	Count occurrences of each unique value in a column.
Feature engineering	Creating new columns from existing ones to improve ML models.

SECTION HW Homework — Submit Before Day 8

Create day7_homework.py, download the Titanic dataset, and push everything to GitHub before midnight. From today onwards, all homework must include real data from a file — no manually typed arrays.

Exercise 1 — Dataset First Look (Beginner)

Load the Titanic CSV. Print: (1) its shape, (2) column names, (3) first 8 rows, (4) last 5 rows, (5) data types of all columns, (6) count of missing values per column as a percentage. Write one sentence of observation for each output.

Exercise 2 — Selecting and Inspecting (Beginner-Intermediate)

From the Titanic dataset: (1) Select only the Name, Age, Sex, and Survived columns. (2) Show the 10th to 20th rows using `iloc`. (3) Get the fare of the passenger at index 100 using `loc`. (4) List all unique values in the Embarked column. (5) Show value counts for the Pclass column.

Exercise 3 — Filtering Passengers (Intermediate)

Write five separate filter queries on the Titanic dataset and print the count for each: (1) All passengers who survived. (2) All female passengers in first class. (3) All passengers under 10 years old. (4) All passengers who paid more than Rs. 5000 ($\text{Fare} * 90 > 5000$). (5) Male passengers over 50 who did NOT survive.

Exercise 4 — Add New Columns (Intermediate)

Add three new columns to the Titanic DataFrame: (1) `fare_inr`: fare in Indian Rupees (multiply by 90). (2) `is_child`: True if Age < 18, False otherwise. (3) `family_size`: `SibSp + Parch + 1` (including the passenger themselves). Then show the first 10 rows of just these three new columns alongside Name.

Exercise 5 — Your Own Dataset (Intermediate)

Go to [kaggle.com](https://www.kaggle.com), create a free account, and download ANY dataset that interests you (start with something small, under 10,000 rows). Load it into Pandas and run all the inspection commands: `shape`, `info`, `describe`, `isnull` `sum`. Write a short paragraph (5 sentences) describing what you found. Push the dataset and your code to GitHub.

Suggested beginner datasets on Kaggle: Iris flowers, IMDB movies, Indian Premier League, Indian food recipes, Zomato restaurant data, COVID India data. Pick something you find genuinely interesting — you will work with it better.

Stretch Challenge — Tier A students

Using the Titanic dataset, answer these business questions using Pandas filtering and Series operations: (1) What percentage of women survived vs what percentage of men survived? (2) Which passenger class had the highest survival rate? (3) What was the average fare paid by survivors vs non-survivors? (4) How many children (under 18) were on board and how many survived? Present your answers as a clean printed report.